

Carnegie Mellon Capstone - Detroit Tigers

DATA-DRIVEN CLUSTERING OF MLB SWING MECHANICS USING HIGH-FREQUENCY MOTION-TRACKING DATA

Technical Specification

Document Header	
Document Name	Data-Driven Clustering of MLB Swing Mechanics Using High-Frequency Motion-Tracking Data An explanation of the pipeline from Hawk-Eye tracking data to cluster classification model
Author	Jason Andriopoulos Malcolm Ehlers Andrew Shih
Created On	May 1, 2025
Object Repository	https://github.com/jasonandriopoulos/mlb-swing-biomechanics-clustering

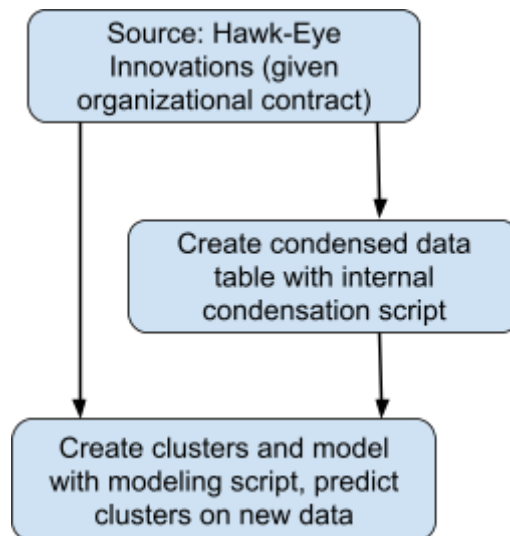
Revision History				
Author	Reason for Revision	TechSpec Version	Date	Current Release Branch
Malcolm Ehlers	Initial Authoring	1.0	May 1, 2025	–
Jason Andriopoulos	Final revisions before initial publishing	1.1	May 2, 2025	

Objects List		
Object Name	Object Type	Object Description
Script to create clusters, model, and predict clusters for new data mlb_swing_clustering.qmd	Quarto document	Creates hierarchical clusters from Hawk-Eye tracking data, creates random forest model to predict clusters from condensed tracking data, and uses random forest model to predict batter clusters from condensed tracking data

High-Level Technical Overview

Obtain parquet files containing tracking data. Create parquet of condensed data using internal script. Separate into training and testing data by batters as desired. Run the appropriate modeling script on the training data and generate predictions on the testing data with the created model.

Process Flow Diagram



Assumptions

- User has access to Hawk-Eye data
- User has data condensation script with particular output variables and calculation methods
- Hawk-Eye does not change their data format or output variables

Dependencies

- `mlb_swing_clustering.qmd` must be opened and run in RStudio or some virtual environment that supports Quarto files.
- Hawk-Eye has data and services readily available.
- RStudio version must be at least RStudio v2022.07.1
- R version must be at least R 4.1.0
- Parquet files containing batter tracking data must be named "batter-X.parquet" where X is an integer
 - o Files must be named uniquely, and must range from 1 to the total number of batters
 - o There must be 100 batters (dependency can be removed with minor edits to code)
 - o Each batter must have 10 swings (dependency can be removed with minor edits to code)

Roles / Authorizations	
Must have access to organization's Hawk-Eye data.	

Program Overview			
Program Name	mlb_swing_clustering.qmd		
Short Text	Creates clusters using UMAP and hierarchical clustering, and creates random forest model from cluster assignments		
Program Type	Quarto notebook	Program Status	Finished
Pseudo Code – Run mlb_swing_clustering.qmd: 1. Fill the location of the folder where your training data is contained into the definition of "folder_path" in the first code chunk 2. Fill the location of the condensed training data frame into the definition of "outcomes_path" in the first code chunk 3. Fill the desired number of final clusters into the definition of "num_swing_clusters" in the nineteenth code chunk 4. Fill the location of the condensed test data frame into the definition of "new_data" in the forty-fifth (final) code chunk 5. Run the first code chunk, which loads the libraries required for the remainder of the file 6. Run the second chunk, which loads the batter tracking data and the condensed data file 7. Run the third chunk, which uses PCA to reduce each joint to a single, maximally informative dimension 8. Run the fourth chunk, which defines these PC dimensions as the variables we use for clustering 9. Run the fifth chunk, which reduces the time window of the data to between the peak gather and contact points of each swing 10. Run the sixth chunk, which isolates the data for the chosen joints, and stores swing IDs for later use 11. Run the seventh chunk, which defines a function to fit a B-spline to a swing's joint's dimension location over time 12. Run the eighth chunk, which fits B-splines to each used joint for each swing 13. Run the ninth chunk, which fills a matrix with the B-spline coefficients for each joint for each swing 14. Run the tenth chunk, which stores the B-splines as functional data objects 15. Run the eleventh chunk, which performs functional PCA on the functional data objects 16. Run the sixteenth chunk, which plots the first three FPCs for each swing 17. Run the seventeenth chunk, which fills a matrix with the FPCA scores for the first three FPCs for each joint for each swing 18. Run the nineteenth chunk, which defines the number of clusters that will be created 19. Run the twenty-third chunk, which averages the swings for each batter, runs UMAP on those swings, and plots the first three UMAP dimensions			

20. Run the twenty-fourth chunk, which confirms the number of clusters that will be created
21. Run the twenty-sixth chunk, which runs UMAP on the batter-level data and clusters them using a Gaussian Mixture Model 250 times
22. Run the twenty-seventh chunk, which aligns each cluster across all runs
23. Run the twenty-eighth chunk, which records the most common cluster for each batter and how often it is assigned to that cluster
24. Run the twenty-ninth chunk, which creates a co-occurrence matrix between the assignments of each batter across all runs
25. Run the thirtieth chunk, which uses hierarchical clustering to assign final clusters to each batter
26. Run the thirty-first chunk, which defines a function to compute the robustness of each cluster between all runs
27. Run the thirty-second chunk, which adds cluster assignment to the batter data and prints a table of cluster assignment and robustness
28. Run the thirty-third chunk, which defines a function to compute cluster consistency for a given batter
29. Run the thirty-fourth chunk, which defines a function to retrieve batters beyond a given consistency threshold
30. Run the thirty-fifth chunk, which creates and displays a set of representative batters for each cluster based on consistency
31. Run the thirty-sixth chunk, which prepares a data frame of summary statistics and cluster assignments for the representative batters
32. Run the thirty-seventh chunk, which fits twenty random forests to the summary statistics predictions cluster assignment, and lists the most important features to fitting these models
33. Run the thirty-eighth chunk, which creates a list of the important features from our run of random forests, sorted by type of feature
34. Run the thirty-ninth chunk, which creates parallel plots of each type of feature, listing the median scaled value of each cluster's representative batters for each feature within the type
35. Run the fortieth chunk, which collects the parallel plots into a list
36. Run the forty-first chunk, which displays the parallel plots and can save them as a PNG if the final line is uncommented
37. Run the forty-second chunk, which displays the distribution of hit results between each cluster's representative batters
38. Run the forty-third chunk, which displays the distribution of swing results between each cluster's representative batters
39. Run the forty-fourth chunk, which fits a random forest on the representative batters, as well as a random forest on all batters, and displays the misclassification rate and a confusion matrix for each
40. Run the forty-fifth chunk, which loads in new test data, aggregates it to batter-level, predicts their cluster classifications using the random forest fit on all the training batters, and displays the predictions